

# MODULE 12 — LOG ANALYSIS

---

## 1. What Is Log Analysis?

A log is a record of an event that happened on a computer, server, application, or network.

Think of a log like the CCTV of a computer system. It can help us answer: Who did something? When did they do it? From where? What did they do?

User logged in

File was modified

SSH connection attempted

Web page was requested

Firewall blocked a connection

Service crashed

Example:

Failed password for root from 192.168.1.50

This tells us that an authentication attempt failed, the target account was root, and the source IP was 192.168.1.50.

## 2. Why Logs Matter in Cybersecurity

Logs are one of the main sources of evidence during a security investigation. Security analysts use them to understand what happened on a system.

| Purpose        | What it means                                                        |
|----------------|----------------------------------------------------------------------|
| Detection      | Find suspicious or abnormal activity.                                |
| Investigation  | Reconstruct what happened, when, and how.                            |
| Threat Hunting | Search proactively for indicators of compromise.                     |
| Compliance     | Support required logging, retention, and review.                     |
| Forensics      | Provide evidence for technical, legal, or regulatory investigations. |

## 3. Security Analyst Mindset

Do not look at one log entry and immediately declare an attack. Treat an entry as evidence and ask what it proves.

Example:

Failed password for root

This proves a failed authentication attempt occurred. It does not by itself prove that a hacker successfully compromised the server.

A stronger pattern is:

03:42 Failed password from 10.10.10.5

03:43 Failed password from 10.10.10.5

03:44 Accepted password from 10.10.10.5

Beginner rule: One event = clue. Multiple related events = pattern. Correlated patterns = investigation.

## 4. Five-Step Log Analysis Process

| Step                  | What to do                                                                                            |
|-----------------------|-------------------------------------------------------------------------------------------------------|
| 1. Scope & Collection | Define the systems and time range. Collect and preserve original logs; preferably investigate copies. |
| 2. Establish Baseline | Understand what normal login volume, users, services, and traffic look like.                          |
| 3. Identify Anomalies | Look for unusual login times, IPs, request volumes, errors, processes, or outbound connections.       |
| 4. Correlate Events   | Connect related events across time and across log sources.                                            |
| 5. Document & Report  | Record timestamps, log files, exact evidence, commands, conclusions, and uncertainties.               |

**Key mindset:** Think chronologically, look for what is missing, correlate across sources, quantify before investigating, and document as you go.

## 5. Major Linux Log Files

| Log / Source            | Common Location / Tool | What it records                                                                                   |
|-------------------------|------------------------|---------------------------------------------------------------------------------------------------|
| <b>syslog</b>           | /var/log/syslog        | General system activity, services, kernel messages, cron, etc.                                    |
| <b>messages</b>         | /var/log/messages      | General system activity on distributions that use this file.                                      |
| <b>auth.log</b>         | /var/log/auth.log      | Authentication, SSH, sudo, su, PAM, and account events.                                           |
| <b>secure</b>           | /var/log/secure        | Authentication and security events on some RHEL/CentOS systems.                                   |
| <b>kern.log</b>         | /var/log/kern.log      | Kernel messages, driver errors, OOM events, and kernel problems.                                  |
| <b>boot.log</b>         | /var/log/boot.log      | Boot/startup information and, depending on configuration, shutdown-related events.                |
| <b>dpkg.log</b>         | /var/log/dpkg.log      | Package installation, removal, and update history on Debian-based systems.                        |
| <b>yum.log</b>          | /var/log/yum.log       | Package activity on systems using <code>yum</code> .                                              |
| <b>cron</b>             | /var/log/cron          | Scheduled task/cron activity on systems that log it here.                                         |
| <b>wtmp</b>             | /var/log/wtmp          | Login/logout history; commonly viewed with <code>last</code> .                                    |
| <b>btmp</b>             | /var/log/btmp          | Failed login attempts; commonly viewed with <code>lastb</code> .                                  |
| <b>lastlog</b>          | /var/log/lastlog       | Most recent login for each account.                                                               |
| <b>systemd-journald</b> | journalctl             | Centralized systemd journal containing system, service, authentication, kernel, and other events. |

## 6. Authentication Log Analysis

Authentication logs record events related to users accessing or attempting to access a Linux system. They are especially important during investigations involving brute-force attacks, unauthorized access, privilege escalation, and suspicious account activity.

On many Debian/Ubuntu systems, authentication events may be stored in:

```
/var/log/auth.log
```

On some RHEL/CentOS systems:

```
/var/log/secure
```

However, these files are not guaranteed to exist. Modern Linux systems, including Kali configurations using systemd-journald, may store authentication events primarily in the system journal.

Use:

```
sudo journalctl
```

## 7. Reading an auth.log Entry

```
May 15 03:42:17 webserver sshd[12847]: Failed password for root from 185.220.101.42 port 54321  
ssh2
```

| Field                 | Meaning                                                         |
|-----------------------|-----------------------------------------------------------------|
| Timestamp             | Date and time of the event. Always clarify the server timezone. |
| Hostname              | The machine that generated the event.                           |
| Service/Process       | sshd and its process ID.                                        |
| Event Type            | For example, Failed password or Accepted password.              |
| Username              | The account being targeted or used.                             |
| Source IP             | The client address that initiated the connection.               |
| Source Port           | The client's source port.                                       |
| Authentication Method | For example, password or public key.                            |

## 8. Authentication Events to Watch

| Pattern                      | Possible meaning                                                                         |
|------------------------------|------------------------------------------------------------------------------------------|
| Failed password              | Repeated failures may indicate brute force or password spraying.                         |
| Invalid user                 | Someone is guessing usernames or using a credential list that does not match the system. |
| Accepted after many failures | Possible successful brute-force activity; correlate with earlier failures.               |
| Unexpected successful login  | May indicate account compromise, especially from an unusual source.                      |
| Accepted for root            | High-priority finding because direct root SSH access should normally be restricted.      |
| sudo ... COMMAND             | Shows a command executed with elevated privileges.                                       |

## 9. Practical Authentication Commands

```
sudo journalctl -u ssh | grep 'Failed password' | wc -l    # Count failed SSH login attempts

sudo journalctl -u ssh | grep 'Failed password' \

| grep -oE '[0-9]{1,3}(\.[0-9]{1,3}){3}' \

| sort | uniq -c | sort -rn | head -10                      # Find top source IPs

last -n 50                                                  # Check login history

lastb -n 50 2>/dev/null                                    # Check failed login history

lastlog | grep -v 'Never'                                  # Check last login for each account

sudo journalctl | grep 'sudo' | grep 'COMMAND='           # Track sudo commands
```

## 10. System Logs: syslog

syslog is a broad system log. On modern Kali/Linux systems, these events may be stored in systemd-journald and viewed with journalctl.

```
sudo journalctl                                # View general system logs

sudo journalctl -n 50                          # View recent system logs

sudo journalctl -k                             # View kernel-related events

sudo journalctl | grep -i 'cron'               # Search for cron activity

sudo journalctl | grep -Ei 'started|stopped|restart' # Search for service restart/start/stop events

sudo journalctl | grep -Ei 'out of memory|oom|killed process' # Search for out-of-memory events

sudo journalctl | grep -Ei 'network|link|interface' # Search for network interface changes

sudo journalctl | grep -Ei 'usb|hardware' # Search for USB/hardware activity

sudo journalctl | grep -Ei 'dns|resolved' # Search for DNS-related activity

sudo journalctl --since "today"                # View logs for a specific time period

sudo journalctl --since "1 hour ago"           # View logs from the last hour
```

## 11. journalctl

Modern Linux systems using systemd commonly store structured logs in the journald journal. journalctl is the command used to query it.

```
journalctl -b

journalctl -u sshd

journalctl -u nginx

journalctl --since '2024-05-15 00:00:00' --until '2024-05-15 23:59:59'

journalctl -p err
```

```
journalctl -f
```

```
journalctl -k
```

Think of journalctl as a powerful search and filtering interface for systemd logs.

## 12. Web Server Logs

Web server logs record HTTP requests. They are useful for identifying normal traffic, scanning, exploitation attempts, suspicious uploads, and unusual application behavior.

| Server | Access Log                  | Error Log                  |
|--------|-----------------------------|----------------------------|
| Apache | /var/log/apache2/access.log | /var/log/apache2/error.log |
| Nginx  | /var/log/nginx/access.log   | /var/log/nginx/error.log   |

## 13. Understanding a Web Access Log

```
203.0.113.42 - jsmith [15/May/2024:09:22:11 +0000] "GET /admin/config.php?debug=true HTTP/1.1" 200 4823
```

| Field         | Meaning                                            |
|---------------|----------------------------------------------------|
| Client IP     | Source address of the HTTP request.                |
| Username      | Authenticated HTTP username if applicable.         |
| Timestamp     | Date/time and timezone.                            |
| Request       | HTTP method, URL/path, query string, and protocol. |
| Status Code   | HTTP result such as 200, 403, 404, or 500.         |
| Response Size | Number of bytes returned.                          |

## 14. Important HTTP Status Codes

| Code | Meaning                           |
|------|-----------------------------------|
| 200  | Request succeeded.                |
| 403  | Forbidden / access denied.        |
| 404  | Requested resource was not found. |
| 500  | Server-side error.                |

Important: a 200 response does not automatically prove an attack succeeded. It only tells us the HTTP request received a successful HTTP response. We need additional evidence to determine the security outcome.

## 15. Common Web Attack Patterns

| Activity                | What to look for                                                                    |
|-------------------------|-------------------------------------------------------------------------------------|
| Directory/Path Scanning | Many 404s and requests for paths such as /admin/, /.env, /.git/config, /backup.zip. |
| SQL Injection Attempt   | SQL-like syntax in parameters, such as UNION SELECT or encoded variants.            |
| XSS Attempt             | HTML/JavaScript patterns such as <script> or javascript:.                           |
| Path Traversal          | Paths containing sequences such as ../../etc/passwd.                                |
| Command Injection       | Shell metacharacters or command-like input in request parameters.                   |
| Web Shell Access        | POST requests to unusual PHP/ASPX files,                                            |

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
|                       | especially in upload or temporary locations.                                |
| Credential Stuffing   | Large numbers of POST requests to login endpoints with varying credentials. |
| Possible Exfiltration | Unusually large response sizes or unexpected large POST bodies.             |

## 16. Web Log Analysis Commands

Top IPs by request volume:

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

Top IPs producing 404 responses:

```
awk '$9==404 {print $1}' access.log | sort | uniq -c | sort -rn
```

Find SQL-injection-like patterns:

```
grep -iE "('%|%27|UNION|SELECT|INSERT|DROP|exec|xp_|0x[0-9a-f])" access.log
```

Find suspicious POST requests to PHP files:

```
grep '"POST.*\.php' access.log | grep -v '/wp-admin\|/wp-login\|/contact'
```

## 17. Web Error Logs

Error logs show what the server could not do and why. They can add context that access logs alone may not provide.

- Repeated file/directory-not-found errors can indicate automated enumeration.
- Repeated PHP fatal errors or exceptions triggered by malformed input may indicate input manipulation.
- Permission-denied errors can indicate probing or misconfiguration.
- Upstream failures may indicate backend crashes or resource exhaustion.
- ModSecurity alerts can directly record detected web attack attempts when the WAF is installed.
- Repeated TLS handshake failures may indicate probing, downgrade attempts, or client misconfiguration.

## 18. Firewall and Network Logs

Firewall logs record network traffic that was allowed or blocked. They complement application logs by showing network activity that may never reach an application.

UFW commonly logs to /var/log/ufw.log and may also write entries into syslog.

```
May 15 03:22:14 webserver kernel: [UFW BLOCK] IN=eth0 OUT= MAC=aa:bb:cc:dd:ee:ff SRC=203.0.113.5 DST=10.0.1.10 PROTO=TCP SPT=54321 DPT=22
```

| Field             | Meaning                                               |
|-------------------|-------------------------------------------------------|
| UFW BLOCK / ALLOW | Whether the firewall denied or permitted the traffic. |
| IN                | Interface where traffic arrived.                      |
| SRC               | Source IP.                                            |
| DST               | Destination IP.                                       |
| PROTO             | Protocol such as TCP or UDP.                          |
| SPT               | Source port.                                          |
| DPT               | Destination port.                                     |

## 19. ss — Current Network Connections

Firewall logs tell us about recorded network events. ss gives a current snapshot of connections and listening ports.

```
ss -tnp
```

```
ss -tlnp
```

Remember: firewall logs help answer 'what happened?' while ss helps answer 'what is happening now?'

## 20. Application Logs

Applications create their own logs. These can reveal business-logic events and application-layer activity that lower-level system logs may not show.

| Application         | Example Log Location                        |
|---------------------|---------------------------------------------|
| MySQL/MariaDB       | /var/log/mysql/error.log                    |
| PostgreSQL          | /var/log/postgresql/postgresql-*.log        |
| PHP-FPM             | /var/log/php-fpm/error.log                  |
| WordPress           | wp-content/debug.log when enabled           |
| Fail2ban            | /var/log/fail2ban.log                       |
| OpenVPN             | /var/log/openvpn.log                        |
| Custom applications | /var/log/<appname>/ or /opt/<appname>/logs/ |

## 21. Building an Incident Timeline

The ultimate goal of log analysis during an incident is a chronological, evidence-based account of what happened.

| Step                 | Action                                                                            |
|----------------------|-----------------------------------------------------------------------------------|
| 1. Anchor Event      | Start with the known alert, detection, or symptom and record its exact timestamp. |
| 2. Expand Backwards  | Search for earlier activity from the same IP, user, or process.                   |
| 3. Expand Forwards   | Trace what happened after access or the suspicious event.                         |
| 4. Correlate Sources | Compare auth.log, access.log, syslog, firewall, and other available logs.         |
| 5. Identify Gaps     | Investigate missing or unusually sparse periods.                                  |

## 22. Example Timeline

| Time  | Event                           | Source       | Significance                                  |
|-------|---------------------------------|--------------|-----------------------------------------------|
| 03:41 | Blocked SSH connection attempts | ufw.log      | Possible reconnaissance/brute-force activity. |
| 03:42 | Failed root login               | auth.log     | Authentication attack reached SSH.            |
| 03:47 | Source IP banned                | fail2ban.log | Automated defensive response.                 |
| 04:13 | Successful authentication       | auth.log     | Critical event requiring investigation.       |
| 04:14 | POST to upload endpoint         | access.log   | Suspicious web activity.                      |

|       |                              |         |                                                                    |
|-------|------------------------------|---------|--------------------------------------------------------------------|
| 04:14 | Cron executes file from /tmp | syslog  | Possible persistence.                                              |
| 04:14 | Outbound connection blocked  | ufw.log | Possible command-and-control or other suspicious outbound traffic. |

The lesson is not to assume every event is malicious. Correlate the events and determine exactly what the evidence supports.

## 23. Log Rotation

Linux rotates logs so that log files do not grow without limit.

```
auth.log
```

```
auth.log.1
```

```
auth.log.2.gz
```

Older logs may be compressed. Use `zgrep` to search compressed log files.

```
zgrep 'Failed password' /var/log/auth.log*
```

```
cat /etc/logrotate.conf
```

```
ls /etc/logrotate.d/
```

During an incident, preserve relevant logs early because rotation can eventually remove older evidence.

## 24. Log Tampering

Log tampering means modifying, deleting, truncating, or otherwise manipulating logs to hide activity.

Possible signs include:

- Unexpected timestamp gaps.
- An active log file suddenly becoming unusually small or empty.
- Unexpectedly missing rotated log files.
- File modification times that do not align with the timestamps of the last legitimate entries.
- Evidence of commands such as `rm`, `shred`, `truncate`, or `echo >` being used to modify files.

```
stat /var/log/auth.log
```

## 25. Protecting Logs

| Control                   | How it helps                                                             |
|---------------------------|--------------------------------------------------------------------------|
| Remote Log Forwarding     | Keeps a separate copy on a central log server.                           |
| SIEM Ingestion            | Stores collected logs independently for monitoring and investigation.    |
| File Integrity Monitoring | Detects unexpected modifications or deletions.                           |
| Append-Only Storage       | Makes modification/deletion of existing log content more difficult.      |
| Log Signing               | Allows integrity verification of signed log records.                     |
| Minimal Permissions       | Prevents compromised application accounts from writing to security logs. |



## 26. Hands-On Beginner Lab

### Scenario: The Quiet Compromise

You are a junior SOC analyst. A monitoring system reports suspicious activity on web-server-01. You are given auth.log, access.log, syslog, and ufw.log. Your task is to determine whether a compromise occurred and reconstruct the evidence-based timeline.

#### Lab Task 1 — Authentication

```
grep 'Failed password' auth.log

grep 'Failed password' auth.log | wc -l

grep 'Failed password' auth.log | grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}'
```

Questions:

- Which IP generated the most failed attempts?
- Which usernames were targeted?
- Did any suspicious IP later successfully authenticate?
- At what exact time did the first suspicious successful authentication occur?

#### Lab Task 2 — Web Investigation

```
grep -E '/admin|/wp-admin|/.env|/.git|/config|/backup' access.log

grep -iE 'UNION|SELECT|INSERT|DROP|%27' access.log

grep 'POST' access.log
```

Questions:

- Which IP performed scanning?
- Which paths were requested?
- Were unusual POST requests made?
- Which requests returned 200, 403, 404, or 500?
- Is there enough evidence to claim successful exploitation?

#### Lab Task 3 — System Investigation

```
grep 'CRON' syslog

grep -E '/tmp|/dev/shm|/var/tmp' syslog

grep 'systemd' syslog
```

Questions:

- Was a suspicious process executed?
- Which user executed it?
- When did it happen?
- Did it occur shortly after a successful login?

#### Lab Task 4 — Firewall Investigation

```
grep 'UFW BLOCK' ufw.log
```

```
grep 'UFW BLOCK' ufw.log | grep -oE 'SRC=[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | sort | uniq -c | sort -rn
```

Questions:

- Which IP was blocked most often?
- Which destination ports were targeted?
- Was the traffic inbound or outbound?
- Does the same IP appear in auth.log?

### Lab Task 5 — Build the Timeline

| Timestamp | Event | Log Source | Evidence | Significance |
|-----------|-------|------------|----------|--------------|
|           |       |            |          |              |
|           |       |            |          |              |
|           |       |            |          |              |
|           |       |            |          |              |
|           |       |            |          |              |
|           |       |            |          |              |

## 27. Student Reporting Format

Your incident report should contain:

| Section                | What to include                                                                                                  |
|------------------------|------------------------------------------------------------------------------------------------------------------|
| Executive Summary      | 3–5 sentences for a non-technical manager: what happened, when, and confirmed impact.                            |
| Technical Timeline     | Chronological events with timestamps, source logs, and significance.                                             |
| Evidence Appendix      | Exact log lines, filenames, line numbers when available, and commands used.                                      |
| Attack Method Analysis | Describe the observed stages: reconnaissance, initial access, execution, persistence, and possible exfiltration. |
| Immediate Actions      | Three specific actions to contain the incident.                                                                  |
| Long-Term Controls     | Three controls that could detect or prevent similar activity earlier.                                            |

## 28. Knowledge Check

1. What is a log, and why is it useful in cybersecurity?
2. Name four Linux log files and explain what each records.
3. What is the difference between a failed login and a successful login?
4. Why should analysts establish a baseline before searching for anomalies?
5. What does the source IP tell you in an authentication log?
6. What is log correlation? Give an example.
7. What is the difference between HTTP 403 and 404?
8. Does a 200 response prove that an attack succeeded? Explain.

9. What is log tampering? Name two possible indicators.

10. Why is it useful to examine auth.log, syslog, web logs, and firewall logs together?

## 29. Essential Command Cheat Sheet

| Purpose                  | Command                     |
|--------------------------|-----------------------------|
| Search text              | grep 'pattern' file         |
| Count matching lines     | grep 'pattern' file   wc -l |
| Extract/transform fields | awk '{print \$1}' file      |
| Sort results             | sort                        |
| Count duplicates         | uniq -c                     |
| Show top results         | head                        |
| Follow a live text log   | tail -f file                |
| Systemd logs             | journalctl                  |
| Current TCP connections  | ss -tnp                     |
| Listening ports          | ss -tlnp                    |
| Login history            | last                        |
| Failed login history     | lastb                       |
| Last login per user      | lastlog                     |
| File metadata            | stat file                   |
| Search compressed logs   | zgrep 'pattern' file.gz     |

## 30. Final Takeaway

Log analysis is not about memorizing thousands of log entries. It is about learning how to ask the right questions from evidence.

Who?

When?

From where?

How many times?

Which account?

Did the activity succeed?

What happened after success?

Was persistence established?

Was there suspicious outbound communication?

Core principle: Don't just look at individual log entries. Connect events together and reconstruct what happened.

## Note

This module in the order: What is a log → why logs matter → five-step methodology → major Linux logs → auth.log → syslog/journalctl → web logs → firewall logs → application logs → timeline → tampering → hands-on investigation.